

Documentação de Arquitetura

Processador Falcon-8 Monociclo com Sistema de Frenagem Inteligente

Matheus Varela de Paula

Maio de 2026

Projeto Falcon-8

Sistema de Controle para Veículos Autônomos

Módulo 1: Sistema de Frenagem Inteligente (SFI)

Logisim Evolution

Processador monociclo de 8 bits

Sumário

1	Introdução	4
2	Características gerais do processador	4
3	Organização da arquitetura	5
4	InstructionFetch	5
5	InstructionDecode	6
6	Banco de Registradores	7
7	ULA	7
8	ImmGen	8
9	Unidade de Controle	8
10	RAM de Dados	10
11	Instruções validadas	10
12	Sistema de Frenagem Inteligente	11
13	ROM final do SFI	12
14	Explicação da ROM do SFI	12
15	Como carregar os testes na RAM	14
16	Casos de teste obrigatórios	14
16.1	Caso 1: Emergência	14
16.2	Caso 2: Operação normal	15
16.3	Caso 3: Alerta	15
16.4	Caso 4: Limite exato	15

16.5 Caso 5: Velocidade alta com distância segura	16
17 Resumo dos resultados	16
18 Problemas encontrados e correções	16
19 Conclusão	17

1 Introdução

Neste projeto, eu desenvolvi um processador monociclo de 8 bits chamado **Falcon-8**, utilizando o ambiente **Logisim Evolution**. O objetivo foi construir uma arquitetura funcional capaz de buscar instruções na memória, decodificá-las, executar operações aritméticas e lógicas, acessar memória de dados e realizar desvios condicionais e incondicionais.

Além do processador, também implementei um programa para o **Sistema de Frenagem Inteligente (SFI)**, que simula uma tomada de decisão em um veículo autônomo. Esse sistema lê valores de sensores armazenados na RAM, processa as condições de distância e velocidade e grava comandos de saída para o freio e para o status do sistema.

Durante o desenvolvimento, eu testei cada unidade separadamente e, depois, validei o funcionamento completo do processador com os casos obrigatórios do Módulo 1.

2 Características gerais do processador

O processador Falcon-8 foi construído com uma arquitetura simples e didática, seguindo o modelo monociclo. Isso significa que cada instrução é executada em um único ciclo de clock, passando pelas etapas de busca, decodificação, execução, acesso à memória e escrita de resultado.

As principais características do processador são:

- Arquitetura monociclo;
- Dados de 8 bits;
- Instruções de 16 bits;
- PC de 8 bits;
- Banco com 8 registradores;
- Registrador R0 fixado em zero;
- ROM de instruções configurada como 256 x 16;
- RAM de dados configurada como 256 x 8;
- Incremento normal do PC em 2 unidades;
- Suporte a instruções aritméticas, lógicas, memória, branch e jump.

Dados:	8 bits
Instrucao:	16 bits
PC:	8 bits
ROM:	256 x 16
RAM:	256 x 8
Registradores:	R0 ate R7
Incremento do PC:	PC + 2

3 Organização da arquitetura

Eu organizei o processador em unidades separadas para facilitar a montagem, os testes e a identificação de erros. As principais unidades implementadas foram:

- InstructionFetch;
- InstructionDecode;
- Banco de Registradores;
- ULA;
- ImmGen;
- Unidade de Controle;
- RAM de Dados;
- Writeback.

Essa separação foi importante porque permitiu testar cada parte isoladamente antes de juntar tudo no datapath principal.

4 InstructionFetch

A unidade **InstructionFetch** é responsável por armazenar o valor atual do PC, calcular o próximo endereço e buscar a instrução correspondente na ROM.

Como as instruções possuem 16 bits e o PC avança de 2 em 2, foi necessário adaptar o endereço enviado para a ROM. Para isso, utilizei a lógica:

```
endereco_ROM = PC >> 1
```

Isso significa que o valor do PC é deslocado uma posição para a direita antes de acessar a ROM. Dessa forma, o PC pode continuar avançando de 2 em 2, enquanto a ROM acessa as instruções de forma sequencial.

A lógica principal do `InstructionFetch` ficou:

```
PC_plus2 = PC + 2
PC_branch = PC + imediato

se PCSrc = 0:
    nextPC = PC_plus2

se PCSrc = 1:
    nextPC = PC_branch
```

O sinal `PCSrc` é gerado pela Unidade de Controle e define se o processador seguirá para a próxima instrução ou se realizará um desvio.

5 InstructionDecode

A unidade **InstructionDecode** recebe a instrução de 16 bits e separa os campos necessários para a execução.

O formato geral utilizado foi:

```
bits 15..12 -> OPCODE
bits 11..9   -> rd
bits 8..6    -> rs1
bits 5..3    -> rs2
bits 2..0    -> FUNCT
```

Também foram gerados os campos de imediato:

```
I_IMM = bits 5..0
S_IMM = bits 5..0
B_IMM = bits 5..0
J_IMM = bits 8..0
```

Durante os testes, percebi que algumas instruções precisavam de ajustes no uso dos campos. Em especial, as instruções de branch precisavam comparar registradores usando campos diferentes. Por isso, adicionei um MUX antes da entrada `rs2` do banco de registradores, permitindo selecionar entre `rs2Decode` e `rdDecode` dependendo do tipo da instrução.

6 Banco de Registradores

O banco de registradores possui oito registradores de 8 bits:

```
R0, R1, R2, R3, R4, R5, R6, R7
```

O registrador R0 foi mantido fixo em zero, funcionando como uma constante. Isso foi importante para permitir instruções como:

```
ADDI x1, x0, valor
LW   x1, deslocamento(x0)
SW   x0, deslocamento(x7)
```

As entradas principais do banco de registradores são:

- **rs1**: seleciona o primeiro registrador de leitura;
- **rs2**: seleciona o segundo registrador de leitura;
- **rd**: seleciona o registrador de escrita;
- **Data**: dado que será escrito;
- **RegWrite**: habilita a escrita;
- **CLK**: clock;
- **RST**: reset.

O banco de registradores foi testado com instruções de carga imediata, operações tipo R, acesso à RAM e desvios.

7 ULA

A ULA foi responsável por executar operações aritméticas e lógicas. As operações implementadas foram:

A operação SLT foi fundamental para o SFI, pois permitiu comparar valores como:

```
DIST_ATUAL < DIST_SEGURA
VEL_MAX_PERMITIDA < VEL_ATUAL
```

A ULA também gera o sinal **ALUzero**, utilizado nas instruções BEQ e BNE.

ALUOp	Operação
000	ADD
001	SUB
010	AND
011	OR
100	SLT

Tabela 1: Operações implementadas na ULA

8 ImmGen

A unidade **ImmGen** gera os imediatos de 8 bits utilizados pelas instruções. Os tipos de imediato implementados foram:

- Tipo I: usado por ADDI e LW;
- Tipo S: usado por SW;
- Tipo B: usado por BEQ e BNE;
- Tipo J: usado por JAL.

A seleção é feita pelo sinal **ImmSel**:

ImmSel	Imediato selecionado
000	Tipo I
001	Tipo S
010	Tipo B
100	Tipo J

Tabela 2: Seleção de imediato

Para instruções de branch e jump, o imediato foi deslocado uma posição para a esquerda:

```
B_8b = B_IMM << 1
J_8b = J_IMM << 1
```

Esse deslocamento foi necessário porque o PC trabalha com instruções de 16 bits, avançando normalmente de 2 em 2.

9 Unidade de Controle

A Unidade de Controle recebe o opcode e gera os sinais responsáveis por comandar o datapath.

Opcode	Instrução
0000	Tipo R
0001	ADDI
0010	LW
0011	SW
0100	BEQ
0101	BNE
0110	JAL

Tabela 3: OpCodes utilizados no Falcon-8

Os opcodes utilizados foram:

Os principais sinais de controle são:

- ImmSel;
- ALUOp;
- ALUSrc;
- MenRead;
- MenWrite;
- MenToReg;
- RegWrite;
- PCSrc.

A lógica dos branches foi uma das partes mais importantes do projeto. Para permitir tanto BEQ quanto BNE, utilizei:

```
Tipo_B_GERAL = Tipo_B OR Tipo_B_BNE

branch_cond = ALUzero XOR Tipo_B_BNE

branch_taken = Tipo_B_GERAL AND branch_cond

PCSrc = Tipo_J OR branch_taken
```

Com essa lógica:

- BEQ desvia quando `ALUzero = 1`;
- BNE desvia quando `ALUzero = 0`;
- JAL sempre desvia.

10 RAM de Dados

A RAM de dados foi configurada como:

```
RAM 256 x 8
Endereco: 8 bits
Dados:      8 bits
```

Ela é utilizada tanto para os testes gerais quanto para o SFI.

As ligações principais foram:

```
Endereco da RAM    <- alu_result
Dado de escrita    <- rd2
Write enable       <- MenWrite
Output enable      <- MenRead
Clock              <- CLK
Saida da RAM       -> ram_data_out
```

O valor escrito de volta no banco de registradores é selecionado por um MUX:

```
MenToReg = 0 -> writeback_data = alu_result
MenToReg = 1 -> writeback_data = ram_data_out
```

11 Instruções validadas

Antes de implementar o SFI, eu testei as instruções principais individualmente e também em conjunto.

As instruções validadas foram:

- ADDI;
- ADD;
- SUB;
- AND;
- OR;
- SLT;
- LW;

- SW;
- BEQ tomado;
- BEQ não tomado;
- BNE tomado;
- BNE não tomado;
- JAL simples.

Também realizei um teste misto com RAM, ULA, branch e jump, confirmando que o datapath principal estava funcionando corretamente.

12 Sistema de Frenagem Inteligente

Depois de validar o processador, implementei o programa do **Sistema de Frenagem Inteligente**. O SFI utiliza a RAM como entrada e saída de dados.

Os endereços de entrada são:

Endereço	Nome	Descrição
0x10	DIST_ATUAL	Distância atual do obstáculo
0x11	VEL_ATUAL	Velocidade atual do veículo
0x12	DIST_SEGURA	Limite de distância segura
0x13	VEL_MAX_PERMITIDA	Velocidade máxima permitida

Tabela 4: Entradas do SFI

Os endereços de saída são:

Endereço	Nome	Descrição
0x20	CMD_FREIO	Comando para o atuador de freio
0x21	STATUS_SISTEMA	Status do sistema

Tabela 5: Saídas do SFI

A lógica implementada foi:

```
Se DIST_ATUAL < DIST_SEGURA:
  Se VEL_ATUAL > VEL_MAX_PERMITIDA:
    CMD_FREIO = 2
    STATUS_SISTEMA = 2
  Senao:
    CMD_FREIO = 1
    STATUS_SISTEMA = 1
```

Senao:

```
CMD_FREIO = 0  
STATUS_SISTEMA = 0
```

13 ROM final do SFI

A ROM final do SFI possui um NOP no início. Esse NOP foi adicionado porque, durante os testes, ele ajudou a estabilizar a primeira leitura da ROM e garantiu a execução correta desde o início.

```
0000  
2210  
2412  
0654  
5606  
1E10  
0FF8  
31C0  
31C1  
6010  
2811  
2A13  
0D64  
4C07  
1E10  
0FF8  
1802  
39C0  
39C1  
6006  
1E10  
0FF8  
1801  
39C0  
39C1  
6000
```

14 Explicação da ROM do SFI

```

0000    ; NOP

2210    ; LW    x1, 0x10(x0)    ; DIST_ATUAL
2412    ; LW    x2, 0x12(x0)    ; DIST_SEGURA
0654    ; SLT   x3, x1, x2      ; x3 = 1 se distancia atual < segura

5606    ; BNE   x3, x0, CRITICO

; NORMAL
1E10    ; ADDI  x7, x0, 0x10
0FF8    ; ADD   x7, x7, x7      ; x7 = 0x20
31C0    ; SW    x0, 0(x7)       ; Mem[0x20] = 0
31C1    ; SW    x0, 1(x7)       ; Mem[0x21] = 0
6010    ; JAL   FIM

; CRITICO
2811    ; LW    x4, 0x11(x0)    ; VEL_ATUAL
2A13    ; LW    x5, 0x13(x0)    ; VEL_MAX_PERMITIDA
0D64    ; SLT   x6, x5, x4      ; x6 = 1 se VEL_MAX < VEL_ATUAL

4C07    ; BEQ   x6, x0, ALERTA

; EMERGENCIA
1E10
0FF8
1802    ; x4 = 2
39C0    ; Mem[0x20] = 2
39C1    ; Mem[0x21] = 2
6006    ; JAL   FIM

; ALERTA
1E10
0FF8
1801    ; x4 = 1
39C0    ; Mem[0x20] = 1
39C1    ; Mem[0x21] = 1

6000    ; FIM

```

15 Como carregar os testes na RAM

Para testar o SFI, eu devo manter a mesma ROM e alterar apenas os valores da RAM.

Na visualização da RAM no Logisim, os endereços importantes aparecem distribuídos da seguinte forma:

```
Linha 0C -> enderecos 0C 0D 0E 0F 10 11
Linha 12 -> enderecos 12 13 14 15 16 17
Linha 1E -> enderecos 1E 1F 20 21 22 23
```

Antes de executar cada caso, eu devo zerar as saídas:

```
Mem[0x20] = 00
Mem[0x21] = 00
```

Depois disso, devo reiniciar o processador:

```
RST = 1
RST = 0
PC = 00
```

16 Casos de teste obrigatórios

16.1 Caso 1: Emergência

Entrada na RAM:

```
0C: 00 00 00 00 02 3C
12: 03 1E 00 00 00 00
1E: 00 00 00 00 00 00
```

Saída esperada:

```
Mem[0x20] = 02
Mem[0x21] = 02
```

Resultado obtido:

```
Mem[0x20] = 02
Mem[0x21] = 02
```

Status: **Aprovado.**

16.2 Caso 2: Operação normal

Entrada na RAM:

```
0C: 00 00 00 00 32 28
12: 03 3C 00 00 00 00
1E: 00 00 00 00 00 00
```

Saída esperada:

```
Mem[0x20] = 00
Mem[0x21] = 00
```

Resultado obtido:

```
Mem[0x20] = 00
Mem[0x21] = 00
```

Status: **Aprovado.**

16.3 Caso 3: Alerta

Entrada na RAM:

```
0C: 00 00 00 00 02 14
12: 03 1E 00 00 00 00
1E: 00 00 00 00 00 00
```

Saída esperada:

```
Mem[0x20] = 01
Mem[0x21] = 01
```

Resultado obtido:

```
Mem[0x20] = 01
Mem[0x21] = 01
```

Status: **Aprovado.**

16.4 Caso 4: Limite exato

Entrada na RAM:

```
0C: 00 00 00 00 03 1E
12: 03 28 00 00 00 00
1E: 00 00 00 00 00 00
```

Saída esperada:

```
Mem[0x20] = 00
Mem[0x21] = 00
```

Resultado obtido:

```
Mem[0x20] = 00
Mem[0x21] = 00
```

Status: **Aprovado.**

16.5 Caso 5: Velocidade alta com distância segura

Entrada na RAM:

```
0C: 00 00 00 00 1E 50
12: 03 32 00 00 00 00
1E: 00 00 00 00 00 00
```

Saída esperada:

```
Mem[0x20] = 00
Mem[0x21] = 00
```

Resultado obtido:

```
Mem[0x20] = 00
Mem[0x21] = 00
```

Status: **Aprovado.**

17 Resumo dos resultados

18 Problemas encontrados e correções

Durante o desenvolvimento do projeto, encontrei alguns problemas importantes.

Caso	Descrição	Esperado	Obtido	Status
1	Emergência	02 / 02	02 / 02	Aprovado
2	Operação normal	00 / 00	00 / 00	Aprovado
3	Alerta	01 / 01	01 / 01	Aprovado
4	Limite exato	00 / 00	00 / 00	Aprovado
5	Velocidade alta com distância segura	00 / 00	00 / 00	Aprovado

Tabela 6: Resumo dos casos de teste obrigatórios

O primeiro foi no endereço da ROM. Como o PC avançava de 2 em 2, mas a ROM acessava posições sequenciais, foi necessário utilizar a lógica $PC \gg 1$.

Outro problema ocorreu nos desvios condicionais. Inicialmente, o BEQ funcionava, mas o BNE não desviava corretamente. A correção foi criar o sinal `Tipo_B_GERAL` e utilizar a lógica com XOR entre `ALUzero` e `Tipo_B_BNE`.

Também foi necessário corrigir o `ImmSel`, pois o BNE precisava usar o mesmo imediato tipo B que o BEQ.

Por fim, durante os testes do SFI, percebi que o programa funcionou de forma estável quando adicionei um NOP no início da ROM. Depois dessa alteração, os cinco casos obrigatórios passaram corretamente.

19 Conclusão

Com este projeto, eu consegui desenvolver um processador monociclo funcional de 8 bits no Logisim Evolution. O processador executa instruções aritméticas, lógicas, acesso à memória, desvios condicionais e salto incondicional.

Além disso, implementei o Sistema de Frenagem Inteligente utilizando a RAM como entrada e saída de dados. O programa do SFI foi capaz de processar corretamente as informações de distância e velocidade, gravando os comandos esperados nos endereços 0x20 e 0x21.

Os cinco casos obrigatórios do Módulo 1 foram testados e aprovados. Dessa forma, considero que o processador Falcon-8 atende aos requisitos obrigatórios do projeto e está pronto para demonstração.